

Report Template

FYS-STK3155 - Project 3

See Gek Cheryl Ong, Boon Kiat Khaw, Naden Jarel Anthony Koh
University of Oslo
(Dated: December 18, 2025)

Breast cancer segmentation plays a key role in diagnostic accuracy and treatment planning in clinical practice. This study evaluates deep learning-based segmentation approaches with a focus on convolutional neural networks (CNNs). We explore how variations in filter configuration, loss functions, and network depth affect segmentation performance, and compare standard convolutional architectures with the U-Net model. The results show that convolutional architectures substantially outperform fully connected baselines under severe class imbalance, with U-Net achieving the best balance between sensitivity and parameter efficiency. These findings highlight the potential of deep learning models to improve the efficiency and reliability of breast cancer image analysis and identify U-Net as a strong candidate for future clinical applications.

I. INTRODUCTION

Breast cancer is one of the most prevalent and life-threatening diseases worldwide, and early detection is often key to increasing patient survival rates. Mammography remains a primary tool for identifying malignancies and assessing tumour boundaries to support treatment decisions. However, manual segmentation of cancerous cells is inefficient, subjective, and requires expert domain knowledge. Automated segmentation therefore presents an important opportunity to improve diagnostic accuracy, efficiency, and consistency.

Deep learning, particularly through convolutional neural networks (CNNs), has demonstrated significant success in medical image analysis. CNN-based segmentation models are capable of learning fine-grained spatial patterns and distinguishing cancerous cells from normal tissue with high accuracy. Unlike traditional manual or rule-based methods, CNNs leverage hierarchical feature extraction to recognise complex visual structures and generalise across large datasets, making them powerful tools for clinical workflows[1].

This study investigates deep learning-based approaches for breast cancer cell segmentation, with a specific focus on CNNs. We first explore the structure and properties of convolutional architectures and discuss why they are well-suited for segmentation tasks compared to fully connected neural networks. To evaluate performance, we conduct systematic experiments involving key hyperparameters such as network depth, filter size, and loss functions.

Beyond standard CNN architectures, we also examine advanced models such as U-Net, which employ encoder-decoder pathways with skip connections to integrate global context with local detail. Through comparative analysis, this report aims to identify how architectural and design choices influence segmentation outcomes and contribute toward more reliable and scalable breast cancer imaging tools.

II. METHODS

A. Method 1/X

1. Problem formulation and data context

The task addressed in this project is *binary pixel-level image segmentation*. Given an input histopathology image

$$X \in \mathbb{R}^{512 \times 512 \times 3},$$

the objective is to predict a segmentation mask

$$Y \in \{0, 1\}^{512 \times 512},$$

where each pixel label indicates whether the corresponding image location belongs to cancerous tissue. This formulation treats segmentation as a dense classification problem, where a class decision is made for every pixel rather than for the image as a whole.

A key challenge in medical image segmentation is *class imbalance*. In histopathology images, cancerous regions often occupy only a small fraction of the total image area. As a result, naïve classifiers may achieve high accuracy by predicting the background class everywhere, while failing to detect tumour regions. This motivates the use of architectures and loss functions that are sensitive to spatial structure and class imbalance.

2. Convolutional neural networks for segmentation

Convolutional neural networks (CNNs) are particularly well-suited for image segmentation tasks because they exploit the spatial locality of images. Unlike fully connected neural networks, CNNs apply convolutional filters that share parameters across spatial locations, drastically reducing the number of trainable parameters and improving generalization.

By stacking multiple convolutional layers, CNNs are able to learn hierarchical feature representations, ranging

from low-level textures and edges to higher-level structural patterns such as tumour boundaries.

In this study, CNNs are used within an encoder-decoder framework. The encoder progressively down-samples the input image to extract abstract feature representations, while the decoder upsamples these representations to recover the original spatial resolution and produce a pixel-wise segmentation mask. This structure enables the model to capture both local texture information and global contextual features.

As segmentation is pixel based, our CNN implements an encoder-decoder architecture for pixel-wise segmentation. The network begins with an input of shape (512, 512, 3) and produces a binary segmentation mask of shape (512, 512, 1).

3. Loss functions and evaluation criteria

To train segmentation models, the network outputs a probability map $\hat{Y}_{ij} \in (0, 1)$ for each pixel, representing the predicted likelihood of belonging to the cancerous class. The choice of loss function plays a crucial role, especially under class imbalance.

Binary cross-entropy (BCE) loss is defined as

$$\mathcal{L}_{\text{BCE}} = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})],$$

and treats each pixel independently. While BCE is widely used, it may favour background predictions when positive pixels are rare.

To mitigate this issue, weighted variants of BCE and overlap-based losses such as the Dice loss are commonly employed. The Dice coefficient measures the overlap between predicted and true masks and directly optimizes segmentation quality rather than pixel-wise accuracy. In this project, multiple loss functions are evaluated to assess their impact on recall, precision, and segmentation robustness.

In addition to binary cross-entropy and its weighted variant, overlap-based loss functions are commonly used in medical image segmentation to directly optimize spatial agreement between predictions and ground truth masks. One such measure is the Dice coefficient, which quantifies the similarity between two sets by comparing their overlap relative to their total size. In the context of segmentation, it measures the overlap between the predicted positive pixels and the true positive pixels.

The Dice loss is defined as one minus the Dice coefficient, such that minimizing the loss corresponds to maximizing the overlap between predicted and ground-truth segmentation masks. Unlike pixel-wise losses, Dice loss is less sensitive to class imbalance, since it depends on the relative overlap rather than the absolute number of background pixels. This makes it particularly suitable for medical imaging tasks where the foreground class occupies only a small fraction of the image.

In practice, Dice loss and binary cross-entropy capture complementary aspects of the segmentation task. Bi-

nary cross-entropy provides stable, pixel-level probabilistic supervision, while Dice loss enforces global structural agreement. For this reason, a combined loss function that linearly blends binary cross-entropy and Dice loss is often used. Such combined losses aim to balance local classification accuracy with global shape consistency, improving robustness in highly imbalanced segmentation settings.

These loss functions are evaluated using precision and recall metrics, which provide clinically meaningful insight into model behavior. Recall is particularly important in breast cancer segmentation, as false negatives correspond to missed cancerous regions and may have severe clinical consequences. Precision is also considered to ensure that predicted positive regions are reliable and do not result in excessive false alarms.

Another loss function designed to address severe class imbalance is focal loss. Focal loss extends binary cross-entropy by down-weighting well-classified examples and focusing training on hard-to-classify pixels. By reducing the contribution of easy background pixels, focal loss encourages the model to allocate more capacity toward learning difficult or rare foreground regions, such as small cancerous structures. This property makes focal loss particularly attractive for highly imbalanced segmentation problems, where the majority class can otherwise dominate the training signal. The focusing mechanism allows the model to dynamically emphasize challenging samples during training, potentially improving sensitivity to under represented classes.

To explicitly account for class imbalance, a weighted variant of binary cross-entropy is also considered. Weighted binary cross-entropy assigns a higher penalty to misclassified positive pixels by scaling the loss contribution of the foreground class. By increasing the cost associated with false negatives, the model is encouraged to predict cancerous regions more aggressively, counteracting the dominance of background pixels in the training data. This approach retains the stability and probabilistic interpretation of standard binary cross-entropy, while improving sensitivity to rare but clinically important positive regions.

4. U-Net architecture

While basic encoder-decoder CNNs are effective, repeated downsampling can lead to loss of fine-grained spatial information, which is critical in medical image segmentation. The U-Net architecture addresses this limitation by introducing *skip connections* between corresponding layers in the encoder and decoder.

U-Net consists of a contracting path that captures contextual information through convolution and pooling, and an expanding path that reconstructs the segmentation map using transposed convolutions. Skip connections concatenate feature maps from the encoder with those in the decoder, allowing high-resolution spatial details to be preserved during reconstruction.

This architectural design is particularly advantageous for histopathology images, where accurate delineation of tumour boundaries is essential. By combining global context with local detail, U-Net has become a standard benchmark model for biomedical segmentation tasks and serves as a central algorithm in this project.

5. Part c: Experimental design and hyperparameter testing

To evaluate the robustness and effectiveness of convolutional segmentation models, a systematic hyperparameter testing procedure is conducted. Model performance in deep learning is highly sensitive to architectural and training choices, such as network depth, number of filters, receptive field size, and optimization parameters. Rather than relying on a single fixed configuration, this study investigates how variations in these hyperparameters influence segmentation behaviour.

Given the limited dataset size and the computational cost associated with training segmentation networks on high-resolution images, a carefully constrained experimental design is adopted. The goal is to identify stable and performant model configurations while maintaining computational feasibility.

The hyperparameter analysis focuses on architectural and optimization parameters that directly affect model capacity and generalization. For convolutional neural networks, the following hyperparameters are investigated:

- **Network depth:** the number of convolutional blocks in the encoder-decoder architecture.
- **Number of filters:** the number of convolutional filters used in the first layer, with subsequent layers scaled proportionally.
- **Receptive field (kernel size):** the spatial extent of convolutional filters, controlling how much local context each layer observes.
- **Learning rate:** the step size used by the optimizer during gradient-based training.

These hyperparameters were selected because they directly influence the bias-variance trade-off. Increasing network depth or filter count improves representational capacity but may lead to overfitting, while overly shallow models may underfit complex spatial patterns present in histopathology images.

Due to the combinatorial explosion of possible hyperparameter configurations, an exhaustive grid search is computationally impractical. Instead, a sequential hyperparameter testing strategy is employed. In this approach, one hyperparameter is varied at a time while all others are held fixed at baseline or previously optimised values.

This sequential strategy allows the marginal effect of each hyperparameter to be analysed independently.

While it does not guarantee a globally optimal configuration, it provides a practical and interpretable method for navigating the hyperparameter space under limited computational resources.

Each candidate model configuration is trained on the training set and evaluated on a held-out validation set. Performance is assessed using segmentation-relevant metrics, including recall, precision, and Dice score, rather than accuracy alone. This choice reflects the severe class imbalance present in breast cancer segmentation tasks, where accuracy can be misleading.

Model selection is based on validation performance, with particular emphasis on recall to minimise false negatives. This prioritisation is clinically motivated, as failing to detect cancerous regions carries significantly higher risk than producing limited false positives.

The hyperparameter testing procedure is closely related to bias-variance considerations. Shallow or narrow networks tend to exhibit high bias, failing to capture complex tumour structures, while deeper or wider networks may suffer from high variance due to the limited dataset size. By systematically varying architectural complexity and observing validation behaviour, the experimental design aims to identify configurations that balance expressive power and generalisation.

B. Implementation

All models were implemented in TensorFlow/Keras [2, 3]. Data handling and visualisation used NumPy, pandas, and Matplotlib [4–6]. Train/test splitting and bootstrap resampling used scikit-learn [7]. Image pre-processing utilised OpenCV [8].

1. Dataset Used

The dataset used in this study was obtained from a publicly available Kaggle repository containing annotated breast cancer cell images [9].

The dataset consists of 58 hematoxylin and eosin (H&E) stained histopathology images of breast tissue, each paired with a manually annotated segmentation mask. H&E staining is routinely applied in clinical histopathology because biological tissues are naturally transparent and lack inherent colour contrast. Hematoxylin binds to nuclei and stains them a deep blue-purple, while eosin stains cytoplasmic and extracellular components pink. This contrast enables pathologists and subsequently machine learning model to distinguish structural and cellular features relevant to diagnosing malignancies.

The images used in this study were sourced from a publicly available repository on Kaggle, where they were originally curated for research in breast cancer cell segmentation. Each image is modified to a resolution of 512×512 .

pixels with three colour channels (RGB), forming an input shape of (512, 512, 3). The corresponding ground-truth segmentation masks delineate cancerous cell regions and serve as the supervisory signal during model training, enabling pixel-level learning for automated segmentation.

2. What is Pixel-Level Segmentation and why we are using them

Pixel-level segmentation involves partitioning an image into meaningful regions by assigning a category label to every individual pixel. However, when dealing with high-resolution images containing a large number of pixels, conventional machine learning classifiers become computationally inefficient for this task. This highlights the need for specialised models capable of capturing spatial information—such as CNNs. CNNs are particularly well-suited for segmentation because they can effectively learn textures, boundaries, and local structural patterns, making them powerful tools for high-resolution image analysis.

3. Algorithm 1: Implementation of Convolutional Neural Networks for Image Segmentation

```
model = Sequential()

# Encoder
model.add(layers.Conv2D(16, (7,7),
activation='relu', padding='same'))
model.add(layers.MaxPooling2D((2,2)))
model.add(layers.Conv2D(32, (7,7),
activation='relu', padding='same'))
model.add(layers.MaxPooling2D((2,2)))

# Decoder
model.add(layers.Conv2DTranspose
(32, (7,7), strides=2, activation='relu',
padding='same'))
model.add(layers.Conv2DTranspose
(16, (7,7), strides=2, activation='relu',
padding='same'))

# Output layer (pixel-wise prediction)
model.add(layers.Conv2D(1, (1,1),
activation='sigmoid'))
```

A CNN comprises stacked convolution layers that learn spatial features through filtering. The encoder extracts hierarchical features such as nuclei boundaries and textures in the segmentation process, while the decoder up-samples features back to their original resolution. This produces a pixel-wise mask. The CNN model used in this project follows an encoder-decoder structure, with two convolution + pooling blocks for feature extraction,

followed by transpose-convolution layers for upsampling (Code Listing ??). The final sigmoid output produces a probability mask for pixel-wise classification.

4. Algorithm 2: Implementation of U-Net

```
# Encoder (contracting path)
for i in range(n_layers):
    filters = n_filters * (2 ** i)
    x = layers.Conv2D(filters, (3,3),
activation='relu', padding='same')(x)
    x = layers.Conv2D(filters, (3,3),
activation='relu', padding='same')(x)
    skip_connections.append(x)
    if i < n_layers - 1:
        x = layers.MaxPooling2D((2,2))(x)

# Decoder (expanding path with skip connections)
for i in range(n_layers - 2, -1, -1):
    filters = n_filters * (2 ** i)
    x = layers.Conv2DTranspose(filters, (3,3),
strides=2, activation='relu')(x)
    x = layers.concatenate([x,
skip_connections[i]])
    x = layers.Conv2D(filters, (3,3),
activation='relu', padding='same')(x)
    x = layers.Conv2D(filters, (3,3),
activation='relu', padding='same')(x)

# Final segmentation output
outputs = layers.Conv2D(1, (1,1),
activation='sigmoid')(x)
```

Unlike plain encoder-decoder CNNs, U-Net introduces skip connections that forward high-resolution features from the encoder to the decoder. This prevents loss of structural detail, which is crucial for medical image segmentation. The U-Net implementation follows a symmetric encoder-decoder architecture (Code Listing 2). The contracting path applies repeated convolution operations to learn hierarchical features, with spatial resolution reduced by max-pooling. The decoder reconstructs the image using transposed convolution and skip connections, which reintroduce high-resolution features from the encoder. This structure is particularly effective for medical image segmentation as it preserves fine tumour boundaries while capturing global context. Our model was trained using binary cross-entropy loss depending on the experiment, with precision, recall and dice score as evaluation metrics.

5. Data loading and preprocessing

To prepare the dataset for training, each histopathology image is paired with its corresponding segmentation mask using matched filenames. Images are resized to a

fixed resolution of 512×512 pixels to ensure consistent tensor dimensions across the dataset. Pixel intensities are normalised to the range $[0,1]$ to stabilise optimisation during training.

Segmentation masks are loaded in grayscale format and converted into binary tensors, where pixel values greater than zero are mapped to the positive class. This preprocessing step ensures that the learning task is formulated as a binary pixel-wise classification problem.

```
image = load_img(img_path, target_size=(512, 512))
image = img_to_array(image) / 255.0

mask = load_img(mask_path, target_size=(512, 512),
color_mode="grayscale")
mask = img_to_array(mask)
mask = (mask > 0).astype("float32")
```

6. Output representation and thresholding

Both the CNN and U-Net models output a per-pixel probability map through a sigmoid activation in the final layer. Each pixel value represents the predicted probability of belonging to the cancerous class.

For evaluation and visualisation, the probability map is converted into a binary segmentation mask by applying a fixed threshold. Pixels with predicted probability above the threshold are classified as positive, producing masks that are directly comparable to the ground-truth annotations.

```
y_pred_binary = (y_pred > 0.5).astype("int32")
```

7. Training setup, loss functions, and evaluation metrics

Model training is performed using mini-batch optimisation with the Adam optimiser. To address class imbalance and clinically relevant error types, different loss functions are explored, including binary cross-entropy, weighted binary cross-entropy, Dice-based losses, combined losses, and focal loss.

Model performance is evaluated using accuracy, precision, recall, and Dice score. Accuracy alone can be misleading in highly imbalanced segmentation tasks; therefore, recall is prioritised to reduce false negatives, while precision is monitored to ensure predicted cancerous regions remain reliable.

```
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy',
            tf.keras.metrics.Precision(),
            tf.keras.metrics.Recall()])
```

```
def dice_loss(y_true, y_pred, smooth=1e-6):
    intersection =
    tf.reduce_sum(y_true * y_pred)
    union = tf.reduce_sum(y_true)
    + tf.reduce_sum(y_pred)
    return 1 - (2. * intersection + smooth)
    / (union + smooth)

def combined_loss(y_true, y_pred):
    return tf.keras.losses.binary_crossentropy
    (y_true, y_pred) \
        + dice_loss(y_true, y_pred)

def focal_loss(y_true, y_pred, gamma=2.0):
    bce = tf.keras.losses.binary_crossentropy
    (y_true, y_pred)
    p_t = y_true * y_pred + (1 - y_true)
    * (1 - y_pred)
    return tf.reduce_mean((1 - p_t) ** gamma * bce)
```

8. Part C

```
for n_layers in layer_list:
    model = build_unet(
        input_shape=(512, 512, 3),
        n_layers=n_layers,
        n_filters=base_filters,
        kernel_size=kernel_size
    )
    history = model.fit(
        X_train, Y_train,
        validation_data=(X_val, Y_val),
        epochs=epochs,
        batch_size=batch_size
    )
```

This loop implements sequential hyperparameter testing by training multiple U-Net models while varying the network depth and keeping other parameters fixed.

```
def build_unet(input_shape, n_layers,
n_filters, kernel_size):
    inputs = Input(input_shape)
    x = inputs
    skip_connections = []

    for i in range(n_layers):
        filters = n_filters * (2 ** i)
        x = Conv2D(filters, kernel_size,
activation='relu', padding='same')(x)
        x = Conv2D(filters, kernel_size,
activation='relu', padding='same')(x)
        skip_connections.append(x)
        x = MaxPooling2D((2,2))(x)

    return Model(inputs, x)
```

This function constructs a U-Net architecture whose depth and capacity can be systematically controlled through input hyperparameters.

```
val_recall = history.history['val_recall']
val_dice = history.history['val_dice']
```

Validation recall and Dice score are tracked to guide model selection under class imbalance, rather than relying on accuracy alone.

C. Use of AI tools

- ChatGPT was used to support code refinement, documentation, drafting and clarification of results.

III. RESULTS AND DISCUSSION

A. Part c: Bias-variance hyperparameter analysis

To quantify how architectural and optimisation choices affect generalisation, we performed a memory-optimised bias-variance analysis using bootstrap resampling. The dataset was split into 46 training images and 12 test images, and each hyperparameter setting was evaluated using 10 bootstraps and 20 training epochs per bootstrap. Model predictions were generated on a fixed subset of the test set for each bootstrap to estimate bias², variance, and total error (defined as bias² + variance).

a. Effect of network depth and number of filters. A moderate network depth consistently produced the lowest estimated total error. In particular, $n_{layers} = 2$ achieved the lowest bias² and overall error among the tested depths, indicating that very shallow models underfit while deeper models did not provide additional benefit under the current training regime. When increasing the number of filters, the estimated bias² decreased as capacity increased, with the best-performing configuration occurring at $n_{filters} = 64$. Improvements beyond this point were not substantial, suggesting diminishing returns once the model has sufficient representational capacity for the task.

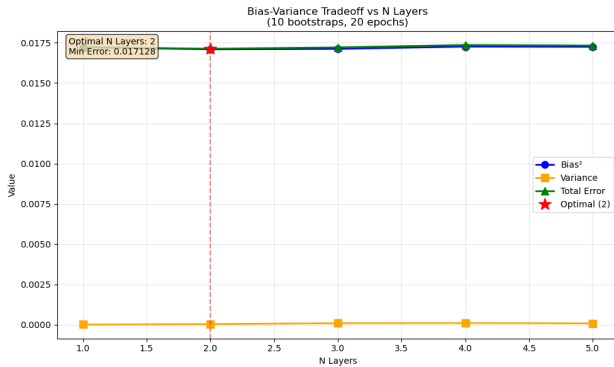


Figure 1: Bias-variance decomposition as a function of network depth (n_{layers}) using 10 bootstrap resamples and 20 epochs per resample. Lower total error indicates better generalisation under the bootstrap protocol.

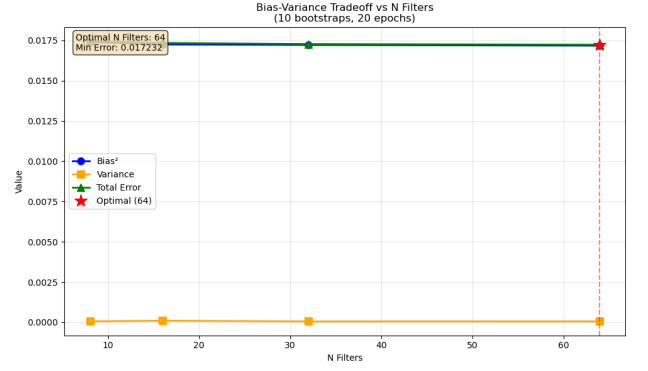


Figure 2: Bias-variance decomposition as a function of model capacity ($n_{filters}$). Increasing filters reduces bias² up to a point, after which gains plateau.

A separate sequential test examined the role of non-linearity and optimization step size. Among the tested activations, ReLU yielded the lowest total error under the bootstrap protocol, indicating a favourable balance between stable optimization and representational power. For the learning rate, the best-performing setting in this experiment was $\eta = 0.001$, while very small learning rates led to substantially higher error, consistent with insufficient optimization progress within the fixed epoch budget.

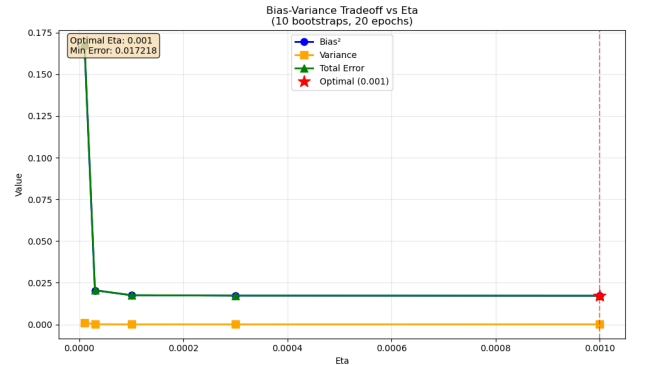


Figure 3: Bias-variance decomposition for different activation functions under fixed training configuration (10 bootstraps, 20 epochs).

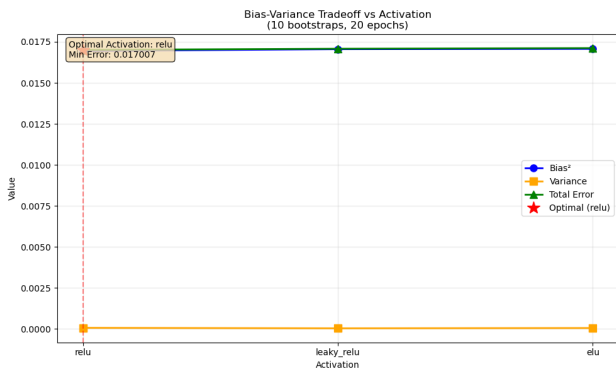


Figure 4: Bias–variance decomposition across learning rates η . Very small learning rates increase error under a fixed training budget, while $\eta = 0.001$ gives the lowest total error in this experiment.

Overall, the bias–variance analysis suggests that the optimal configurations under the tested settings were moderate depth and moderate-to-high capacity. In particular, the sequential depth–capacity experiment selected $n_{layers} = 2$ and $n_{filters} = 64$, while the optimization study selected activation = ReLU and $\eta = 0.001$. These values are carried forward as the default configuration for subsequent training and evaluation experiments in Parts d and e.

B. Part D

This section presents and discusses the segmentation performance of different neural network architectures evaluated in this study. In particular, we compare a fully connected baseline model with a convolutional encoder–decoder CNN and a U-Net architecture. Performance is assessed using loss, accuracy, recall (sensitivity), and precision, with recall prioritized due to the severe class imbalance inherent in pixel-level cancer segmentation tasks.

All models are trained and evaluated under the same data split and training protocol described previously, enabling a direct comparison of architectural effects on segmentation quality and generalization.

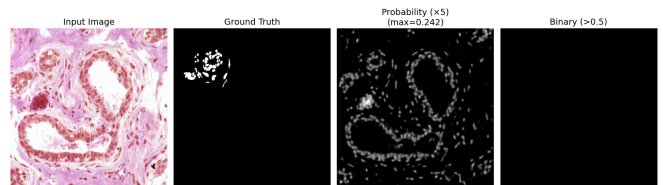


Figure 5: Qualitative segmentation example showing (from left to right) the original histopathology image, the manually annotated ground-truth mask, the predicted probability map (scaled by a factor of five for visualization), and the binarized prediction using a threshold of 0.5. While tumor-related spatial structures are visible in the probability map, low confidence values result in an empty binary mask after thresholding, highlighting the impact of class imbalance and fixed threshold selection in pixel-level segmentation.

Figure 5 illustrates a representative segmentation example, including the original histopathology image, the ground-truth mask, the predicted probability map, and the binarized prediction. The ground truth highlights the extreme sparsity of cancerous pixels relative to the background, confirming the strong class imbalance present in the dataset.

While the probability map reveals that the model has learned meaningful spatial structures corresponding to tumour regions, the predicted probabilities remain low across the image. As a result, applying a fixed threshold of 0.5 produces an empty segmentation mask. This demonstrates that although the network captures relevant features, confidence calibration is poor under standard loss functions, and naive thresholding can obscure meaningful predictions.

This behavior motivates the use of alternative loss functions and evaluation metrics, such as Dice-based losses and recall, which are more appropriate for highly imbalanced segmentation tasks.

Table I compares validation performance across different loss functions under identical training conditions. While standard binary cross-entropy (BCE) and focal loss achieve high validation accuracy, both exhibit zero recall, indicating complete failure to detect cancerous pixels. This highlights the misleading nature of accuracy in highly imbalanced segmentation tasks, where predicting only background pixels can yield superficially strong performance.

In contrast, weighted binary cross-entropy and Dice-based losses substantially improve recall by explicitly penalising false negatives and accounting for class imbalance. Among the tested losses, weighted BCE achieves the highest validation recall while maintaining stable prediction magnitudes, as evidenced by the number of pixels exceeding the decision threshold and moderate mean predicted probabilities. Dice loss attains comparable recall but exhibits more extreme probability values, suggesting less stable calibration.

These results motivate the selection of weighted BCE as the primary loss function for subsequent experiments, as it provides the most favourable trade-off between sen-

Table I: Validation performance comparison across different loss functions, sorted by validation recall. In addition to standard metrics, the maximum and mean predicted probabilities and the number of pixels exceeding a threshold of 0.5 are reported to illustrate confidence calibration under severe class imbalance.

Loss Function	Val Loss	Val Recall	Val Precision	Val Accuracy	Max Pred	Mean Pred	Preds (> 0.5)
Weighted BCE	0.3138	0.3906	0.1106	0.9345	0.7578	0.1016	193881
Dice	0.8277	0.3893	0.1092	0.9339	1.0000	0.0633	195846
Combined Dice+BCE	0.4769	0.1532	0.1264	0.9667	0.8824	0.0321	66566
BCE	0.0718	0.0000	0.0000	0.9825	0.3280	0.0153	0
Focal (high recall)	0.0104	0.0000	0.0000	0.9825	0.4584	0.1606	0

sitivity and prediction stability under severe class imbalance.

A key difference between the evaluated architectures lies in their parameter efficiency. The fully connected baseline model requires approximately 5.37×10^8 trainable parameters due to flattening the high-resolution input image, resulting in a dense mapping between pixels. This dramatically increases model capacity without incorporating any spatial inductive bias.

Despite achieving high accuracy, the fully connected model exhibits extremely poor recall, indicating that it largely predicts background pixels and fails to identify cancerous regions. This behaviour is expected under severe class imbalance, where accuracy can be maximized by ignoring the minority class entirely.

In contrast, the convolutional encoder-decoder CNN and the U-Net architecture exploit spatial locality through parameter sharing, reducing the number of trainable parameters by several orders of magnitude. The U-Net model, in particular, achieves competitive performance using only approximately 1.03×10^5 parameters while substantially improving recall compared to the fully connected baseline.

The standard convolutional encoder-decoder CNN exhibits intermediate performance between the fully connected baseline and U-Net, benefiting from convolutional parameter sharing but lacking the skip connections that enable U-Net to preserve fine spatial detail.

Layer (type)	Output Shape	Param #
flatten_9 (Flatten)	(None, 786432)	0
dense_27 (Dense)	(None, 512)	402,653,696
dense_28 (Dense)	(None, 512)	262,656
dense_29 (Dense)	(None, 262144)	134,479,872
reshape_9 (Reshape)	(None, 512, 512, 1)	0

Total params: 537,396,224 (2.00 GB)
Trainable params: 537,396,224 (2.00 GB)
Non-trainable params: 0 (0.00 B)

Loss	0.4166	1.0111
Accuracy	0.9544	0.9443
Recall (Sensitivity)	0.1490	0.0865
Precision	0.0692	0.0366
Total time:	288.65 seconds (4.81 minutes)	
Time per epoch:	5.77 seconds	

Figure 6: Model summary and evaluation metrics for the fully connected baseline architecture. Flattening the high-resolution input image results in approximately 5.37×10^8 trainable parameters, leading to high apparent accuracy but extremely poor recall. This illustrates how dense architectures without spatial inductive bias perform poorly under severe class imbalance, despite large model capacity.

Layer (type)	Output Shape	Param #
conv2d_180 (Conv2D)	(None, 512, 512, 16)	2,368
max_pooling2d_92 (MaxPooling2D)	(None, 256, 256, 16)	0
conv2d_181 (Conv2D)	(None, 256, 256, 32)	25,120
max_pooling2d_93 (MaxPooling2D)	(None, 128, 128, 32)	0
conv2d_transpose_92 (Conv2DTranspose)	(None, 256, 256, 32)	50,208
conv2d_transpose_93 (Conv2DTranspose)	(None, 512, 512, 16)	25,104
conv2d_182 (Conv2D)	(None, 512, 512, 1)	17

Total params: 102,817 (401.63 KB)

Trainable params: 102,817 (401.63 KB)

Non-trainable params: 0 (0.00 B)

Loss	0.2587	0.3037
Accuracy	0.9305	0.9333
Recall (Sensitivity)	0.5442	0.4135
Precision	0.1227	0.1134

=====
Training completed!
Total time: 281.18 seconds (4.69 minutes)
Time per epoch: 5.62 seconds
=====

Figure 7: Model summary and evaluation metrics for the U-Net architecture. By exploiting convolutional parameter sharing and spatial locality, U-Net achieves effective segmentation performance using approximately 1.03×10^5 trainable parameters. Compared to the fully connected baseline, the model demonstrates substantially improved recall while maintaining competitive accuracy, highlighting the efficiency of task-specific architectural design.

The superior performance of U-Net can be attributed to its architectural inductive bias. Convolutional layers preserve spatial structure and exploit local correlations in the image, while skip connections in U-Net allow high-resolution features from early layers to be directly propagated to the decoder. This mitigates information loss caused by downsampling and improves boundary localization in the segmentation output.

Compared to a standard encoder-decoder CNN, U-Net demonstrates improved sensitivity to small tumour regions, reflected in higher recall values. This highlights the importance of incorporating task-specific architectural design, rather than relying solely on model capacity, when addressing dense prediction problems such as medical image segmentation.

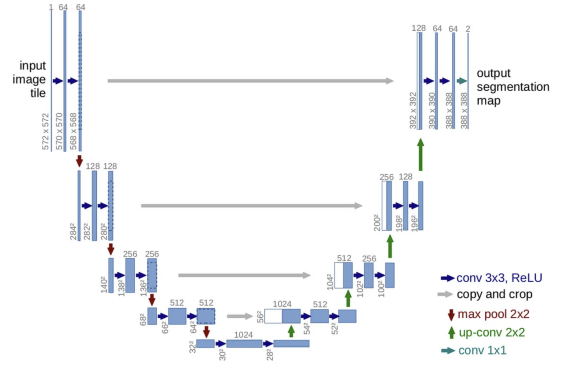


Figure 8: Schematic illustration of the U-Net architecture. The encoder path extracts hierarchical features through successive convolution and pooling operations, while the decoder reconstructs spatial resolution using upsampling. Skip connections transfer high-resolution features from the encoder to the decoder, preserving spatial detail and improving boundary localisation in segmentation tasks. Adapted from Alejandro Ito Aramendía (2023) [10].

While the U-Net architecture significantly outperforms the fully connected baseline and standard CNN in terms of recall and parameter efficiency, several limitations remain. The dataset consists of only 58 images, which restricts the statistical robustness of the evaluation and increases sensitivity to data splits. In addition, segmentation performance is strongly influenced by threshold selection and loss function choice, indicating that probability calibration remains a challenge.

Furthermore, the reported results are based on a limited number of epochs and a fixed training budget, which may prevent full convergence for certain configurations. Future work could explore data augmentation, adaptive thresholding strategies, and more extensive cross-validation to further improve segmentation robustness and generalizability.

IV. CONCLUSION

This study investigated deep learning-based approaches for breast cancer image segmentation, with a focus on convolutional architectures and their ability to address severe class imbalance in pixel-level prediction tasks. A fully connected baseline, a convolutional encoder-decoder CNN, and a U-Net architecture were evaluated under a consistent experimental setup to assess the impact of architectural design, loss functions, and hyperparameter choices on segmentation performance.

The results demonstrate that model architecture plays a critical role in segmentation quality. The fully connected baseline, despite its extremely large parameter count, exhibited poor generalization and very low recall, highlighting the limitations of dense models without spatial inductive bias. In contrast, convolutional architectures achieved substantially better performance

with orders of magnitude fewer parameters, underscoring the importance of parameter sharing and spatial locality. Among the tested models, U-Net consistently achieved the best balance between sensitivity and efficiency, benefiting from skip connections that preserve high-resolution spatial information during decoding.

Hyperparameter analysis revealed that moderate architectural complexity provides the most favourable bias–variance trade-off under the given computational constraints. Increasing depth and capacity beyond this regime did not yield proportional improvements and, in some cases, led to diminished returns. These findings emphasize that effective segmentation is not achieved through model size alone, but through carefully designed architectures aligned with the structure of the data.

An important observation from the bias–variance analysis is that the bias and variance curves did not intersect within the tested hyperparameter range. This suggests that the models may not have reached a sufficiently high complexity regime for variance to dominate, and that further increases in depth or capacity could potentially reduce bias at the cost of increased variance. However, exploring this regime was constrained by computational limitations, as training deeper or wider models on high-resolution images incurred prohibitive memory and run-time costs.

Practitioners applying these models in different set-

tings are therefore encouraged to extend the bias–variance analysis under less restrictive computational budgets. In particular, evaluating higher-capacity architectures, longer training schedules, or alternative regularization strategies may reveal the full bias–variance trade-off and enable more precise model selection for specific deployment contexts.

Despite these promising results, several limitations remain. The dataset used in this study is relatively small, consisting of only 58 images, which restricts the statistical robustness of the evaluation and increases sensitivity to data partitioning. In addition, segmentation performance is strongly influenced by loss function choice and threshold selection, indicating ongoing challenges in probability calibration under extreme class imbalance. The fixed training budget and absence of cross-validation further limit the generalizability of the reported metrics.

Future work could address these limitations by incorporating data augmentation, adaptive or learned thresholding strategies, and more extensive cross-validation protocols. Exploring hybrid loss functions and calibration-aware training objectives may further improve robustness. More broadly, integrating domain-specific priors and evaluating models on larger, multi-centre datasets would be essential steps toward deploying deep learning-based segmentation tools in real clinical workflows.

-
- [1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, *Proceedings of the IEEE* **86**, 2278 (1998).
 - [2] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, et al., in *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)* (2016).
 - [3] F. Chollet et al., *Keras*, <https://keras.io> (2015), accessed 18 December 2025.
 - [4] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, et al., *Nature* **585**, 357 (2020).
 - [5] W. McKinney, in *Proceedings of the 9th Python in Science Conference (SciPy 2010)* (2010), pp. 56–61.
 - [6] J. D. Hunter, *Computing in Science & Engineering* **9**, 90 (2007).
 - [7] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, et al., *Journal of Machine Learning Research* **12**, 2825 (2011).
 - [8] G. Bradski, Dr. Dobb’s Journal of Software Tools (2000).
 - [9] A. M. van der Walt, *Breast cancer cell segmentation*, <https://www.kaggle.com/datasets/andrewmvd/breast-cancer-cell-segmentation> (2018), kaggle dataset.
 - [10] A. I. Aramendía, *Decoding the u-net: A complete guide*, <https://medium.com/@alejandro.itoaramendia/decoding-the-u-net-a-complete-guide-810b1c6d56d8> (2023), accessed 18 December 2025.